דוח משוב לסטודנט

מזהה סטודנט	N102431485
מספר מבחן	0001
תאריך בחינה	יום חמישי, 8 במאי 2025
מזהה קורס	20594
שם קורס	מערכות הפעלה
מרצה	אריה לויטן

תשובות נכונות (שאלות סגורות)	ניקוד שאלות סגורות	ניקוד שאלות פתוחות	ציון מבחן מקורי	ציון מבחן סופי
5	25.00	57.00	82.00	**82.00**

סיכום

מספר שאלה בטופס זה	מספר השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור	ניקוד	ניקוד מירבי	התשובה שסומנה	התשובה הנכונה
1	1	3	5.00	5.00	3 ✓	3
2	2	1	5.00	5.00	1 ✓	1
3	3	1	5.00	5.00	1 ✓	1
4	4	4	0	5.00	4 ✗	2
5	5	2	5.00	5.00	2 ✓	2
6	6	2	5.00	5.00	2 ✓	2
7	7		15.00	25.00		
8	8		16.00	16.00		
9	9		14.00	14.00		
10	10		5.00	5.00		
11	11		5.00	5.00		
12	12		2.00	5.00		
13	13			5.00		

✓ תשובה נכונה · ✓ תשובה חלקית · ✗ תשובה שגויה · ⊘ לא נענתה · ⓘ תשובה עודפת

האוניברסיטה הפתוחה

דף תשובות

מספר גרסה	מספר תעודת זהות	מספר הקורס	תאריך בחינה
0001	[redacted]	2059[redacted]	080525

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימון, יש לסמן X על התשובה הלא נכונה - ולהשאיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה, ולהשאיר מחדש את סימן ה- באופן שיכסה המחיקה -

תשובה \ שאלה	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
א		●	●																	
ב					●	●														
ג	●																			
ד				●																
ה																				
ו																				

תשובה \ שאלה	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
א																				
ב																				
ג																				
ד																				
ה																				
ו																				

תשובה \ שאלה	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
א																				
ב																				
ג																				
ד																				
ה																				
ו																				

שאלון 512 ג 98.20.4 M3

[redacted]
מספר תעודת זהות
יש לרשום את כל תשע הספרות

ש מ [redacted]

מספר
[redacted] ת.ז.

האוניברסיטה הפתוחה

י' באייר תשפ"ה

8 במאי 2025

מס' שאלון - 512

מס' מועד 98

סמסטר 2025א

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **13** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלק א' (שאלות אמריקאיות) עליכם לבחור בכל פעם
בתשובה יחידה הנכונה ביותר מבין התשובות.
את התשובות בחלק א' יש לסמן בטופס המחשב המצורף לשאלון.

ג. בחלקים ב' ו ג' מופיעות שאלות פתוחות.
ענו בגוף השאלון תשובות מלאות, בכתב קריא ובקיצור נמרץ.
אין חובה להשתמש בכל השורות המוקצות לצורך התשובות,
אך אין לחרוג מהמקום המוקצה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

מבחן או"פ 2025א מועד ב

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלק א' (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק א' יש לסמן בטופס המחשב המצורף לשאלון.**
2. בחלקים ב' ו ג' מופיעות שאלות פתוחות. ענו בגוף השאלון **תשובות מלאות**, **בכתב קריא ובקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולסמן את התשובה **בטופס המחשב המצורף לשאלון.**

שאלה 1

מה הבדל בין קישור סימבולי(SOFT LINK) וקישור קשיח(HARD LINK)?

א. יצירת SOFT LINK דורשת יצירת קובץ חדש ויצירת HARD LINK אינה דורשת.

ב. פתיחת הקובץ דרך HARD LINK איטית יותר מאשר דרך SOFT LINK

ג. מחיקת ה HARD LINK יכולה לגרום למחיקת הקובץ המקושר ומחיקת ה SOFT LINK לא.

ד. יצירת SOFT LINK דורשת עדכון מספר המצביעים בתוך ה I-NODE של הקובץ ויצירת HARD LINK אינה דורשת.

נא לא לכתוב בשוליים

שאלה 2

מה נכון לגבי HYPERVISOR מסוג 1 בתצורת bare metal ?

א. כל משאבי החומרה hardware resources נשלטים **באופן ישיר** ע"י hypervisor ולא ע"י מערכת הפעלה.

ב. רק הקצאות זיכרון ראשי ופעולות קלט-פלט עוברות דרך hypervisor , אבל הקצאת זמן המעבד CPU הפיזי מבוצעת **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ג. רק הקצאות זיכרון ראשי עוברות דרך hypervisor , אבל פעולות קלט-פלט מבוצעות **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ד. קוד של כל מערכת הפעלה שרצה על hypervisor צריך לעבור שינוי והתאמה כדי שתוכל לרוץ.

שאלה 3

רוצים לאפשר העברת מידע בין תהליכונים שונים (לא שייכים לאותו תהליך אב). מתלבטים בין שימוש בזיכרון משותף (shared memory) לבין העברת הודעות (massage passing) באמצעות מערכת ההפעלה.

מה ניתן לומר על השיקולים לבחירה בין שתי האפשרויות הנ"ל?

א. אם מדובר בהעברת מידע יחידה(שמתבצעת פעם אחת), לא כדאי ליצור אזור זיכרון משותף להעברת הנתונים.

ב. כאשר ישנן מספר רב של העברות מידע, שימוש בהעברת הודעות ע"י מערכת ההפעלה מהיר יותר מהעברת אותה כמות נתונים ע"י זיכרון משותף.

ג. שתי אפשרויות העברת המידע פועלות באותה המהירות – כיוון ששתיהן מערבות את מערכת ההפעלה ונדרשת החלפת הקשר עבור כל העברת מידע.

ד. העברת מידע באמצעות מערכת ההפעלה מתאימה רק להעברת מידע בין תהליכונים משותפים לאותו תהליך אב, לא בין תהליכים שונים.

נא לא לכתוב בשוליים

שאלה 4

במערכת UNIX נתונה הורצו 2 התכניות הבאות:

```c
//Program 1
#define AR_SIZE 2048
main() {
        int  i , j ,a[ AR_SIZE ][ AR_SIZE ];
        for ( i=0; I < AR_SIZE; i++ )
                for ( j=0; j < AR_SIZE;  j++ )
                        a[ i ][ j ]=0;
}
```

```c
//Program 2
#define AR_SIZE 2048
main() {
        int  i , j , a[ AR_SIZE ][ AR_SIZE ];
        for ( i=0; I < AR_SIZE; i++ )
                for ( j=0; j < AR_SIZE;  j++)
                        a[ j ][ i ]=0;
}
```

שימו לב, ההבדל היחיד בין התכניות הוא בשורה אחת בלבד(השורה האחרונה). שתי התכניות הורצו במקביל(בו-זמנית) מספר רב של פעמים ובכל פעם נמדד הזמן מתחילת ההרצה עד לסיומה. התברר כי בכל פעם התכנית הראשונה מסתיימת בזמן קצר יותר מאשר התכנית השנייה. ידוע שגודל המערך הדו-ממדי a נכנס לקבוצת העבודה. מה הסיבה **הסבירה ביותר** לכך?

הערה: שימו לב שעקב מספרן הרב של ההרצות ניתן לומר שהתכניות נמצאות בתנאים זהים מבחינת הקצאת משאבים על-ידי מערכת הפעלה.

א. הרצת התכנית השנייה גורמת ליותר פעולות החלפת דפים בזיכרון.

ב. הרצת התכנית השנייה גורמת לתחלופה תדירה יותר של תוכן ה-cache במעבד.

ג. הרצת התכניות במקביל עלולה לגרום לקיפאון ולכן מערכת ההפעלה מעדיפה לסיים את הרצת התכנית הראשונה קודם.

ד. התשובות א ו-ב הן הנכונות.

שאלה 5

מה מבין האפשרויות הבאות מהווה יתרון כאשר משתמשים ב-user threads (תהליכונים הממומשים במרחב המשתמש) ולא ב תהליכוני גרעין?

א. חסימת תהליכון אחד לא תגרום לחסימת כל התהליך.

ב. החלפה(context switch) בין תהליכונים של אותו תהליך מהירה יותר.

ג. קיימת אפשרות יצירת משתנים לוקאליים לתהליכון.

ד. קיימת אפשרות לשלוט על כמות הזמן שכל תהליכון מקבל.

נא לא לכתוב בשוליים

שאלה 6

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים :

שורות = גודל בלוק / כתובת = $\frac{2^{12}}{2^2} = 2^{10}$

- גודל הבלוק במערכת הקבצים הוא 4 KBytes 2^{12}
- כתובת הבלוק היא 4 בתים (bytes) 2^2
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

חשבו כמה בלוקים בסה"כ (כולל בלוקים של מצביעים, אבל לא כולל ה I-NODE עצמו) יתפוס קובץ שהגודל הנתונים שלו 12328 KBytes .

הערה : לצורך החישוב 1 Kbyte = 1024 Byte .

א. 3072.

ב. 3082. (מוקף)

ג. 3095.

ד. 3096.

המשך הבחינה בעמוד הבא

10 בלוקים ⇐ 10×4 = 40KB

1 בלוק ⇐ 0KB (single)

1024 בלוקים ⇐ 1024×4 = 4096KB

3 בלוקים ⇐ 0KB (double)

1024 בלוקים ⇐ 1024×4 = 4096KB

1024 בלוקים ⇐ 1024×4 = 4096KB

10+1+1024+3+1024+1024 = 3086

שאלון 512 98.20.4 M3

נא לא לכתוב בשוליים

חלק ב (55 נקודות)

ענו על שאלות 7-9.

שאלה 7 (25 נקודות)

דמיינו בית קפה עם 9 מקומות ישיבה.

- אם תגיעו כשיש מקום ריק, תוכלו לשבת מיד.
- אבל אם תגיעו כשכל 9 המושבים מלאים, זה אומר שכולם סועדים ביחד במסיבה, ותצטרכו לחכות עד **שכל הסועדים** יעזבו לפני שתיישבו.

ניתנו המשתנים הבאים וקוד סועדים בבית קפה כפתרון אפשרי.

```
int  eating = 0 , waiting = 0  // מספר אנשים יושבים או מחכים בבר
semaphore  mutex = 1 // סמפור בינארי המאותחל ל 1
semaphore  block = 0  // סמפור מונה המאותחל ל 0
bool  must_wait = false
// eater process תהליך של אוכל
down(mutex)
if (must_wait) {
     waiting += 1
     up(mutex)
     block.wait ()
     down(mutex) // re-acquire mutex תפוס מחדש את
      waiting -= 1
     }
eating += 1
if (eating == 9){must_wait = true}
else{must_wait = false}
up(mutex)
// eat sushi אוכל סושי
down(mutex)
eating -= 1
if (eating == 0) {
     n = min (9 , waiting )
     up(block , n)
     must_wait = false
     }
mutex.signal()
```

(המשך השאלה בעמוד הבא)

מניעה הדדית
מניעת הרעבה
מניעת קיפאון

נא לא לכתוב בשוליים

א. (10 נק') באלו מהדרישות המתוארות לעיל ודרישות כלליות לפתרון בעיית קטע קריטי הקוד הזה עומד? הסבירו את התשובה שלכם.

האלגוריתם עומד בדרישת המניעה ההדדית באמצעות מנעול mutex שבכל פעם שרוצים לאכול נועלים אותו וכשמסיימים הקטע הקריטי פותחים אותו ובכך מונעים מכמה סועדים להיכנס ביחד. בנוסף, במידה ויש 9 אנשים שסועדים ביחד קיים לנו גם משתנה בוליאני של must_wait שברגע שיש את הכמות המקסימלית מסומן להיות true עד שלא יכנס עוד סועד לקטע הקריטי מאחר ויש מקסימום סועדים. בנוסף, האלגוריתם עומד גם בכך שלא יוכלו להתיישב במידה ויש תפוסה מלאה ועד שכל ה-9 יקומו כי

יכולים להיכנס יותר מ 9

רק כשמגיעים ל-eating==0 משחררים את wait ונותנים לסועדים להיכנס. האלגוריתם מונע קיפאון כי ברגע שמסיימים לאכול מורידים את כמות האוכלים עד שהמונה מגיע ל-0 ומכניסים את המחכים בתור ואין מצב בו יש אפס סועדים ואנשים ממתינים שזה מצב קיפאון במקרה שלנו.

ב. (15 נק') תקנו את הקוד כדי שיעמוד בכל הדרישות המתוארות בשאלה ובכל הדרישות לפתרון בעיית קטע קריטי. אפשר להשתמש במספרי השורות שבקוד ולציין מה צריך להוסיף/למחוק/לשנות.

ההסבר אינו נכון

את שורה 14 בקוד אחרי שיוצאים מהקטע הקריטי יש לשנות ל-~~up(mutex)~~ ולא ל-down כדי לאפשר לאחרים להיכנס לקטע הקריטי אחרי שהוא יצא.

לא פותר את הכל

ובשורה 12 שלפני הקטע הקריטי יש לבצע down(mutex) כדי למנוע מסועדים אחרים (אפשר למחוק את השורה כי בשורה 1 כבר מבצע down(mutex)) שלא צריכים להיות בקטע הקריטי בזמן הזה להיות שם. בשורה 21 כמו mutex.signal() אבל זה סמפור שעושים עליו פעולות up, down. בנוסף בשורה 5 עשינו block.wait() מבלי לעשות block.signal() דבר שעלול לגרום להרעבה כי כל המחכים ימשיכו להמתין לנצח אם לא נעיר אותם. לכן צריך להוסיף שורה בקוד של block.signal() במידה ו-eating==0+1. בנוסף, יש לעדכן את המונה block כי במידה ונכנסו "ממתינים" לשבת אז המונה צריך להתעדכן ולרדת ולא להתעדכן רק כאשר נכנסים ל-if(eating==0), אז לדעתי צריך אחרי שורה 15 שהורדנו את מס' האוכלים ב-1 אחרי הקטע הקריטי גם לבצע block-=1 לעדכן את המונה. כל העדכונים הללו מבטיחים שבמידה והמקום ריק נוכל להתיישב במידי לאכול, דבר שגם מונע קיפאון והרעבה, מבטיח שלא יכנסו יותר מהתפוסה המלאה של הסועדים כי משגילים את כמות הסועדים ל-9 ורק כשמגיע ל-0 מאפשרים למחכים להיכנס, מבטיחים מניעה הדדית באמצעות הורדת ה-mutex שמסמל רצון לכניסה לקטע הקריטי והעלאה שלו במידה ויצאנו מהקטע הקריטי על מנת לא ליצור הרעבה עבור תהליכים שממתינים להיכנס. בנוסף הערת block באמצעות פקודת signal מבטיחה גם מניעת הרעבה כי אם לא הייתה מבוצעת הסועדים הממתינים היו ממתינים לנצח ולא נכנסים לקטע הקריטי. כלל השינויים יבטיחו שהקוד יעמוד גם בדרישות השאלה ובכל הדרישות לפתרון בעיית קטע קריטי וביניהם היעדר הרעבה, היעדר קיפאון ומניעה הדדית.

אני סוליבנית בסימסטר האחרון שלה אז ממש שיקלו עליי בבדיקה, אני רוצה ציון גבוה לסוף התואר :) ♡

נא לא לכתוב בשוליים

שאלה 8 (16 נקודות) 2^{12}

במערכת עם זיכרון וירטואלי ודפים בגודל 4 KB נתונה טבלה הבאה עבור תהליך P (המספרים הם בינאריים):

	מספר מסגרת הזיכרון Page frame number	סיבית הנוכחות Present bit	#השורה(רק סימון מספר שורה, אינו חלק מן הטבלה)
0	0010	0	0
1	1000	(1)	1
2	1011	1	2
3	1010	(0)	3
4	0101	1	4
	1001	1	5
	1100	0	6
	0010	1	7

את התשובות יש לכתוב כמספר בינארי.

(6 נק') א. אילו ערכים או תוצאות יחזיר ה-MMU עבור שתי הכתובות הוירטואליות הבאות של גישה לזיכרון ראשי(אורך הכתובת הוירטואלית במערכת אינה סטנדרטית):

$1\times2^0=1$ 001100010010010

$1\times2^0+1\times2^1=3$ 011100110011001

תשובה והסבר:

גודל הדף הנתון הוא $2^{12}=4KB$ ולכן ה-Offset בשתי הכתובות הוא 12 הביטים התחתונים. האינדקס בטבלה של הכתובת הראשונה הוא 1, וסיבית הנוכחות שלו דלוקה (1) ולכן הכתובת שה-MMU תחזיר היא שילוב של ה-offset והמספר מסגרת ונקבל: 1000100010010010

האינדקס של הכתובת השנייה הוא 3, אם סיבית הנוכחות שלו כבויה (0) ולכן יקרה page fault ונצטרך להביא אותו מהדיסק.

(3 נק') ב. האם הזמנים שיידרשו למערכת ההפעלה לבצע את שתי הדרישות בסעיף א' זהים? נמקו.

תשובה והסבר:

הזמנים שיידרשו למערכת ההפעלה לבצע את שתי הדרישות אינם זהים כי הכתובת הראשונה כבר נמצאת בזיכרון לעומת הכתובת השנייה שמצריכה page fault שזו פעולה יקרה שלוקחת יותר זמן.

98.20.4 M3 שאלון 512

נא לא לכתוב בשוליים

(4 נק') ג. ומה גודלו של מרחב הזיכרון הווירטואלי המרבי האפשרי של התהליך במערכת אם אורך כתובת וירטואלית כמו בטבלה?

תשובה והסבר:

גודלו של הזיכרון הוירטואלי האפשרי הוא מס' השורות כפול ה-offset,
האינדקס מסומן ע"י 3 ביטים
, במירבי ניתן שיהיה במקסימאלי 111,
משמע 8, שורות, ולכן גודלו המירבי של מרחב הזיכרון הוירטואלי
יהיה: $2^{12} \times 2^{3} = 2^{15}$ Bytes

(3 נק') ד. מה הוא הגודל המרבי של הזיכרון הפיזי במערכת הנ"ל אם אורך שדה מספר מסגרת כמו בטבלה?

16
(8)

תשובה והסבר:

גודלו המרבי של הזיכרון הפיזי במידה ואורך המסגרת כמו בטבלה
יהיה אורך מסגרת כפול גודל הדף, אורך המסגרת הוא 4
ולכן נקבל כי גודלו המירבי יהיה:
$2^{4} \times 2^{12} = 2^{16}$ Bytes

נא לא לכתוב בשוליים

שאלה 9 (14 נקודות)

התבוננו באיור להלן שמציג את תוכן קבצי התיקיות /usr/user1 ו /usr/user2 במערכת קבצים מבוססת I-NODE :

/usr/user2

.	26
..	12
bin	31
memo	70
f.c	59
prog1	38

/usr/user1

.	36
..	12
mail	16
games	81
test	10
note	70

(5 נק') א. הסבירו מה משמעות המספרים בטבלה. הסבירו מה המשמעות שמספר 70 מופיע פעמיים.

תשובה והסבר :

משמעות המספרים בטבלה הם ה-I-node שנוצר בעבור ם קבצי התיקיה והמס' 70 מופיע פעמיים כתוצאה מיצירת קישור קשיח Hard-link שמצביע לקבצים.

(4 נק') ב. הסבירו מה המשמעות של "." ו ".." המופיעים בשורות העליונות בכל טבלה. איזו סוג קובץ הם מייצגת ומה שמם?

תשובה והסבר :

המשמעות של "." זה מייצג את התיקיה הנוכחית שבה נמצאים, בצד ימין התיקיה הנוכחית היא /usr/user2 ובצד שמאל התיקיה הנוכחית היא /usr/user1
המשמעות של ".." זה מייצג את תיקיית האב של התיקיות שנמצאות כאן שבמקרה שלנו תיקיית האב של שתי התיקיות הוא /usr

(5 נק') ג. בתוך user1 הוחלט ליצור תת תיקיה בשם new. ציירו טבלה שתייצג את התיקיה הריקה שנוצרה וכתבו את השורה שצריך להוסיף בטבלה של /usr/user1 (במקרה הצורך בחר שרירותית מספר דו-ספרתי בהתחשבות לנתונים הקיימים).

ציור :

/usr/user1

.	36
..	12
mail	16
games	81
test	10
note	70
new	75

/usr/user1/new

.	75
..	36

14
(9)

הסבר :

הנחתי שמס' ה-I-node של התיקיה החדשה new זה 75,
בצד ימין ניתן לראות את האיור שמציג את תוכן התיקיה /usr/user1
כולל תיקיית new החדשה שלה מספר I-node שהוא 75. בצד שני ניתן לראות איור שמציג את תיקיית ה-new כאשר "." מייצג את התיקיה עצמה (הנוכחית) עם ה-I-node שלה שהוא 75 ואת ".." שמייצג את תיקיית האב של התיקיה new שבמקרה הזה הוא /usr/user1 עם ה-I-node שלו שהוא 36 לפי סעיף א'.

נא לא לכתוב בשוליים

חלק ג (15 נקודות, 5 נקודות לכל שאלה)

יש לענות על **שלוש שאלות**. שאלות 10 ו- 11 מתאימות לכלל הסטודנטים, שאלות 12 ו-13 הן שאלות מותאמות על פי הסמסטר בו למדתם.

5
(10)

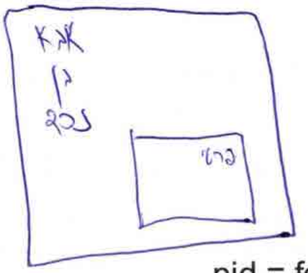
שאלה 10

וקטור הפסיקות

האם לכל קריאת מערכת חייב להיות מספר?

תשובה והסבר:

קריאות המערכת ממוספרות כדי שיהיה קל למערכת ההפעלה לזהות את וקטור הפסיקות ולבצע את הפעולה הנדרשת, לכל קריאת מערכת חייב להיות מספר על מנת שיהיה קל לזהות אותה ולבצע אותה במהירות גבוהה יותר אך אם לא היה מספר לקריאת המערכת היא עדיין הייתה יכולה להיות מטופלת אבל זהו דבר פחות יעיל שהיה דורש יותר זמן לביצוע הפעולה, מה שנחסך ע"י מספור קריאת המערכת.



שאלה 11

מה יהיה הפלט אחרי הרצת הקוד הבא במערכת XV6 שתומכת בקונטיינרים?

ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת הקוד היה 20 .

ניתן להניח שכל קריאות מערכת מצליחות.

	PID אמיתי	PID משתנה
אבא	20	21
בן	21	22
נכד	22	0

```
pid = fork();
if (pid == 0) {  // בן
   pid = fork();
   unshare(PID_NS);
   if (pid > 0) printf (stdout, "Parent's perspective on his child: PID=%d \n",
pid);
   else printf (stdout, " Child's perspective on his parent: PID=%d \n",
getppid());
   }
else printf(stdout, "Grandparent's perspective on the child. PID=%d \n", pid);
```

5
(11)

תשובה והסבר:

בשורה הראשונה מתבצע fork() שמחזירה לאבא את ה-PID של הילד שהוא 21 (נתון שהPID הגדול ביותר לפני היה 20) והילד מקבל 0 ולכן נכנס ל-if. הבן מבצע fork() ונוצר נכד, הבן מקבל את ה-PID של הנכד שהוא 22 והנכד מקבל 0. אחרי זה הבן עושה unshare. הבן נכנס שוב ל-if ומדפיס parent's perspective on his child: PID=22 ואז הנכד יכנס ב-else וידפיס child's perspective on his parent PID=21 שזה ה-PID האמיתי של הילד שמוגדר כערך מ-getppID(). בנוסף האבא יכנס ב-else וידפיס Grandparent's prespective on the child. PID=21

נא לא לכתוב בשוליים

שאלה 12 מיועדת רק לסטודנטים מסמסטר א 2025

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
int pid = fork();
if (pid == 0) { // בן
  unshare (MOUNT_NS);
  mkdir ("dirS");
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
 }
if (pid > 0) sleep (10000); // wait for child to finish
if( pid == 0) { // בן
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to
open\n");
   else printf (stderr, "son succeeded\n");
 }
If (pid>0) { // אבא
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to
open\n");
   else printf (stderr, "father succeeded\n");
```

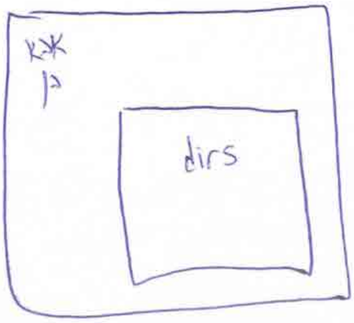
תשובה והסבר:

בשורה הראשונה מתבצע fork() שמחזיר לאבא את ה-PID של הילד/ילדה (שונה מ-0) ולילד מחזיר 0. לכן ה- if נכנס הילד ומבצע unshare(mount_ns) ואז מייצר תיקייה בשם dirS ופותח קובץ file.txt, האבא בינתיים ישן, הבן נכנס שוב ל- if ומנסה לפתוח את הקובץ ומצליח כי נמצא ב-namespace בו יצר את התיקייה והקובץ ומדפיס son succeeded ואז האבא נכנס ב- if בסוף, מנסה לפתוח את הקובץ אבל נמצא במרחב הגלובלי ולא יכול לגשת אליו ולכן ידפיס ~~father failed to open~~

לא היה MOUNT

2
(12)

המשך הבחינה בעמוד הבא



	PID משתנה	PID אמיתי
אבא	1	1
בן	0	2

נא לא לכתוב בשוליים

שאלה 13 מיועדת רק לסטודנטים מ 2024

צוות מתכנתים עובד על הוספת תמיכה בקונטיינרים למערכת דמוית LINUX שלא תמכה בהם לפני זה. אחרי הוספת המנגנות יצירת הקונטיינרים(קריאות מערכת חדשות, כמו UNSHARE למשל) והגדרת המשאבים שלהם, האם יהיה צורך לשנות את קוד מקור של **קריאות מערכת** קיימות?

אם עניתם לא, תסבירו איך קריאות המערכת הקיימות ימשיכו לעבוד בצורה תקינה.

אם עניתם כן, תסבירו בצורה כללית ותנו דוגמא אלו קריאות מערכת יהיה צורך לשנות ומה השינוי העיקרי הנדרש באופן ביצוען.

הערה: התשובה הכללית מדי בסגנון "צריך להתחשב בקונטיינרים" לא תתקבל כי לא מהווה הסבר.

תשובה והסבר:

בהצלחה!

נא לא לכתוב בשוליים